
THE EVOLUTION OF MACHINE LEARNING AND AI

Week 4 Core Booklet

Three Ways to Wire a Machine: the required reading, in plain English

Module	The Evolution of Machine Learning and AI (5 ECTS, Semester 1)
Week	Week 4 of 12
Learning outcomes	LO2: Characterise the training of the main families of model · LO3: Compare and contrast the main model architectures and their assumptions
Assessed in	Week 6 quiz (50% of the module mark). Everything the quiz can ask about Week 4 is in this booklet.
Reading time	About 45 minutes. Read it once, slowly, after the lecture and before the lab.
Goes with	The Week 4 lecture · the six-exercise lab, one exercise per chapter · SUBMISSION.md

What this booklet is. The **required** reading for Week 4, short on purpose. Same six parts as the lecture, same order, same words. Every red ★ **For the Quiz** box is a statement the Week 6 quiz can test. Grey **Extra** boxes say where the optional, deeper account lives in the **full** booklet; you are not required to read the full booklet. No maths beyond add, subtract, multiply, and two formulas, both explained in words first.

Contents

1 A guess about the world	3
2 Convolution: things near each other	5
3 What the guess buys, and when it does not pay	8
4 Recurrence: one thing after another	10
5 Attention: who should talk to whom	13
6 The bargain	16
7 The whole week on one page	18
8 Twenty practice questions, with answers	20

YOUR WEEK, IN ORDER

1	The lecture (or its recording).	2 h
2	This booklet. Once, slowly.	45 min
3	The lab: six exercises, one per chapter of this booklet, in the same order.	6 × 20 min
4	Check SUBMISSION.md (already filled in) against your run; upload it.	10 min
+	Optional: the browser toys, the “break it” challenges, the full booklet.	only if you want

About five hours. If an exercise takes more than 40 minutes, stop and ask in the channel.

1 A guess about the world

COVERS Lecture Part 1 · Lab Exercise 1

TIME 8 minutes

IN BRIEF How you wire a machine is a guess about your data. A true guess is worth a mountain of examples; a false one is worse than no guess. Every learner needs a guess of some kind, because examples alone never choose an answer.

WORDS IN THIS CHAPTER, IN PLAIN ENGLISH

wiring	which parts of the machine are connected to which
dials	the numbers inside the machine that training adjusts (Week 3's parameters)
a guess, inductive bias	an assumption about the world, built in before any data is seen
flexible	able to draw many different answers; not the same as good
chance	the score you get by guessing at random; with four kinds of picture, 0.25
the shuffle test	scramble the pixels the same way in every picture and see which model cares

1.1 Three ways to run an office

Imagine three ways of running an office. In the **grid office**, everyone talks only to the desks touching theirs, and everyone follows the same photocopied rulebook. In the **corridor**, one long line of desks passes a note from one to the next and the last desk answers. In the **open room**, anybody may talk to anybody, and people say what they have and what they need so the right pairs find each other. Those are the three families of machine in this week: **convolution, recurrence, attention**. None is the right way to run an office; which works depends on where the information sits.

1.2 A puzzle you cannot solve without a guess

1 goes to 2, 2 goes to 4, 3 goes to 6. What does 4 go to? You said 8. But “double it, unless it is 4, then a thousand” fits the three examples just as well, and so do infinitely many other rules. When you picked “double it” you used a preference that was not in the examples: you like simple rules. That preference is called the **inductive bias**, and every learner, human or machine, must have one, or it has no way to say what comes next.

1.3 A tiny model beats a big one

Two machines learn to sort small pictures into four kinds. One is tiny, 700 dials, wired with a built-in guess: “pixels near each other belong together, and a pattern worth finding in one corner is worth finding in every corner”. The other is big, 16,708 dials, wired with no guess at all. From the same 128 pictures:

model	dials	score on pictures it studied	score on new pictures
tiny, with the guess	700	0.992	0.945
big, no guess	16,708	1.000	0.275 (chance is 0.250)

The big one was not undertrained: it scored 1.000 on what it studied. It was starved of **reasons to prefer one answer over the others**. Give it 95 times the dials and 128 times the pictures and it finally catches up. That is the **exchange rate**: one true guess was worth about 128 times the data.

IN PLAIN ENGLISH

A more flexible model is not a better model. With little data, a machine that can draw a million boundaries has no reason to draw the right one. This is Week 3's memorising with a new cause: capacity pointed in no particular direction.

1.4 The shuffle test

Take one fixed random shuffle of the 256 pixels and apply the same shuffle to every picture. Nothing is lost; you could un-shuffle it; the labels are still perfectly knowable. The only thing destroyed is which pixels are neighbours.

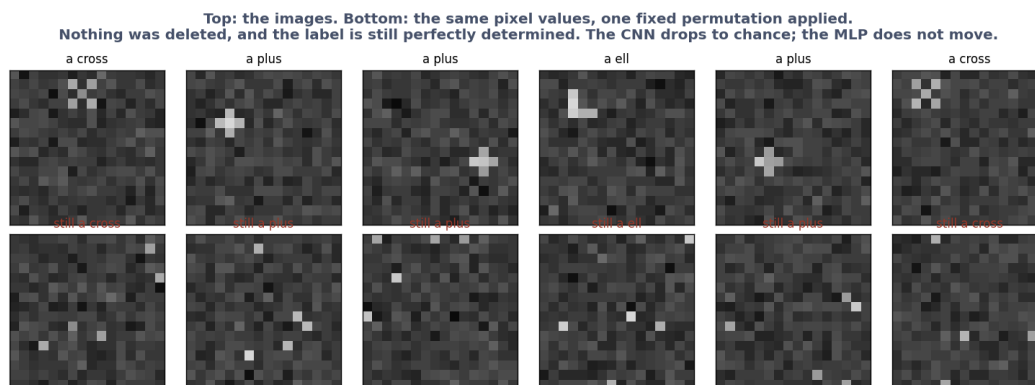


Figure 1: The same picture before and after one fixed shuffle (from the lab). Every pixel is still there; only “who is next to whom” is gone.

pixel order	tiny model	big model	gap
original	0.992	0.328	+0.664
shuffled	0.255	0.333	-0.078

The big model did not move; it never cared about neighbours. The tiny model fell to chance. Its whole advantage was one guess, and when the guess became false it was **worse** than assuming nothing.

★ FOR THE QUIZ

- Any set of examples fits **infinitely many** rules. Choosing one needs a preference not in the data: the **inductive bias**. Every learner has one.
- How a model is **wired** is a guess about the world. A true guess means fewer dials, less data, better results on new examples.
- A more flexible model is **not** a better model: 700 dials beat 16,708 by 67 points at 128 pictures, and the big one had 1.000 on training.
- The **shuffle test**: destroy only the thing the guess is about. If the model falls to chance, its advantage was the guess.

EXTRA · OPTIONAL · NOT ON THE QUIZ

Full booklet Chapters 1 to 5: the formal statement, no free lunch, weight sharing as a constraint, and how to read architecture notation.

2 Convolution: things near each other

COVERS Lecture Part 2 · Lab Exercise 2

TIME 8 minutes

IN BRIEF A convolution is a stencil of nine numbers slid over a picture. Multiply the pairs, add, write one number; move the stencil; the nine numbers never change. That single fact is most of what a convolution is.

WORDS IN THIS CHAPTER, IN PLAIN ENGLISH

kernel	a small square of numbers, a stencil, slid over the picture
convolution	multiply the stencil by the pixels under it, add up, write one number, slide, repeat
feature map	the grid of numbers that comes out: where the stencil's pattern was found
weight sharing	the same stencil numbers used at every position
locality, stationarity, compositionality	the three long words for the three plain sentences in this chapter
equivariance	shift the picture and the answer shifts the same way, exactly
dense layer	the opposite of a stencil: every pixel has its own separate wire to every output

2.1 Watch the kernel walk

A **kernel** is a little square of numbers, usually three by three. Lay it over the top-left corner of the picture so nine pixels sit under nine numbers. Multiply each pixel by the number on top of it, add up the nine results, write down that one number. Slide the stencil one step right. **The nine numbers do not change.** Multiply, add, write. Keep going until the output fills in. That output is a **feature map**: a map of where the stencil's pattern was found, big numbers where it matched and small numbers where it did not.

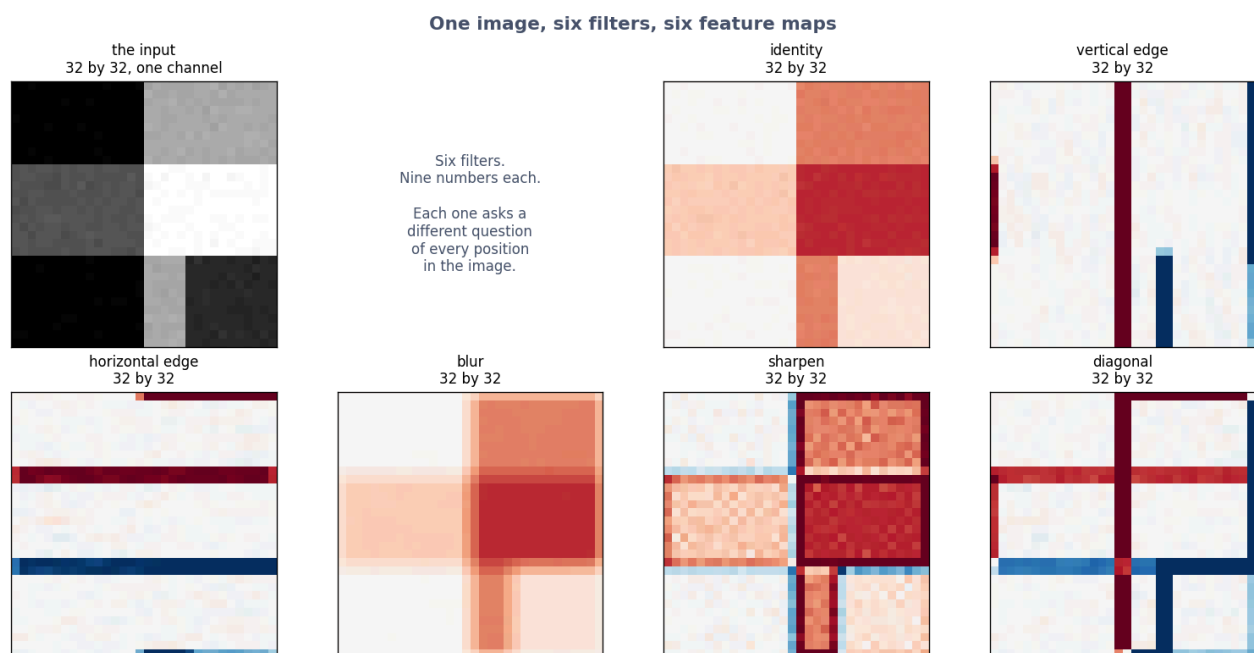


Figure 2: Six different kernels slid over the same picture (from the lab). Each feature map is a map of **where** that kernel's pattern is.

2.2 A convolution by hand

A patch of picture, dark on the left and bright on the right, and a kernel built to find left-to-right edges:

$$\begin{pmatrix} 0.1 & 0.2 & 0.9 \\ 0.1 & 0.2 & 0.9 \\ 0.1 & 0.2 & 0.9 \end{pmatrix} \times \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

Multiply each pair and add, row by row: $-0.1 + 0 + 0.9 = 0.8$; $-0.2 + 0 + 1.8 = 1.6$; $-0.1 + 0 + 0.9 = 0.8$. Total **3.2**: a big number, an edge is here. Now a flat patch, every value 0.5: the answer is **0**, because the kernel's nine numbers add up to zero, so a flat region gives no response. This is a detector for **change**, not brightness. Turn the patch round and the answer is **-3.2**: the sign says which way round the edge is.

2.3 What a convolution assumes

Three sentences about the world:

- **Things near each other belong together.** What a pixel means is decided by its neighbours. True for photos; false for a spreadsheet, where column order means nothing. (**Locality.**)
- **A pattern worth finding here is worth finding everywhere, and it looks the same.** That is why the nine numbers do not change as they slide. True for a cat's ear. (**Stationarity, or weight sharing.**)
- **Big things are made of small things.** Edges make corners, corners make parts, parts make objects; so you stack layers. (**Compositionality.**)

2.4 Move the picture and the answer moves with it

Shift the picture four pixels right, then convolve; or convolve, then shift the result. You get **exactly** the same thing.

Convolution commutes with translation. The bottom right panel is empty, and that is the point.

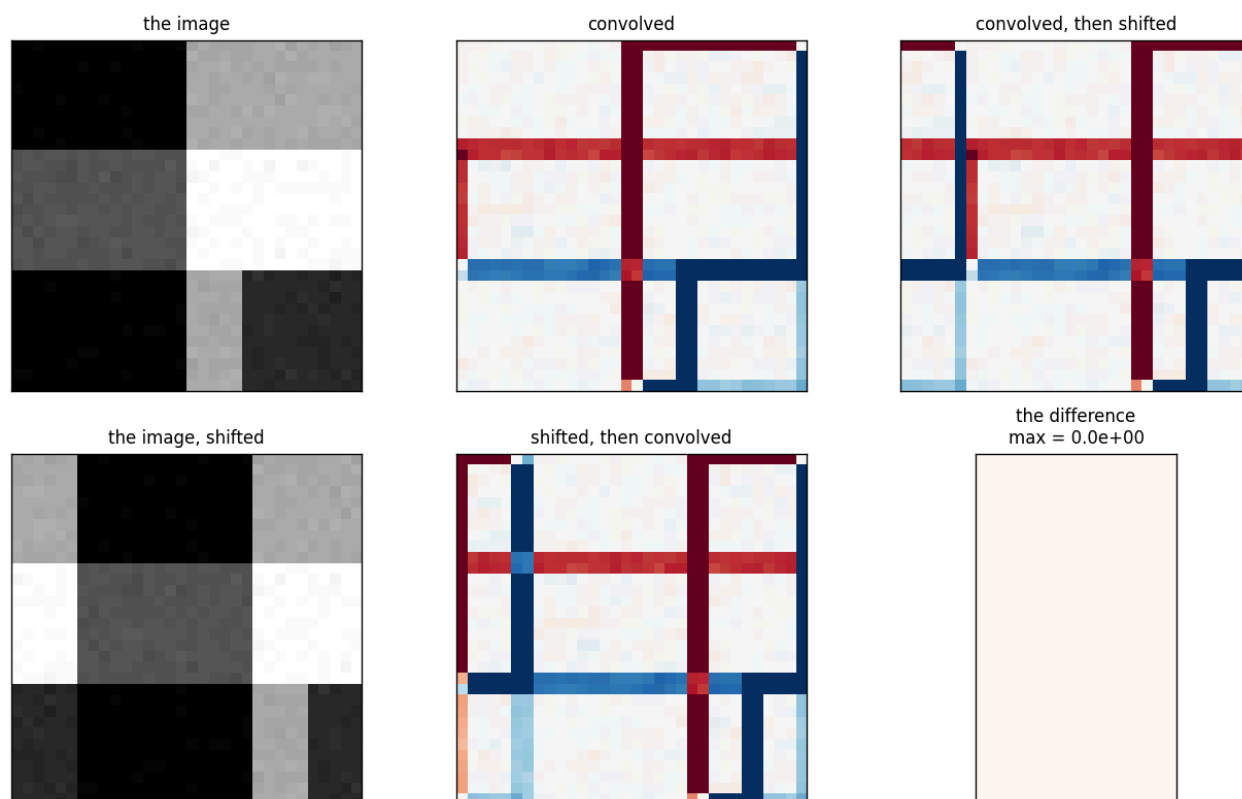


Figure 3: Shift then convolve, and convolve then shift (from the lab). The difference is 0.00e+00.

operation	difference between the two orders	verdict
convolution	0.00e+00	identical, by construction

dense layer (every pixel wired to every output)	1.987	unrelated: it sees a new picture
---	--------------	----------------------------------

This is **equivariance**: move the input and the output moves the same way. A convolution has it because the same nine numbers are used everywhere, so it cannot tell where it is. A dense layer has a different wire for every pixel, so a shifted picture is simply a different picture to it.

★ FOR THE QUIZ

- A **convolution** slides a small square of numbers (a **kernel**) over the picture: multiply the pairs, add. The kernel does not change as it moves: **weight sharing**.
- It assumes **locality** (nearby things belong together), **stationarity** (a pattern here is worth finding everywhere) and **compositionality** (big things are made of small things).
- **Equivariance**: shift the picture and the output shifts the same way, exactly (0.00e+00). A property of the arithmetic, not something learned.
- The output is a **feature map**: a map of where the kernel's pattern was found.

EXTRA · OPTIONAL · NOT ON THE QUIZ

Full booklet Chapters 6 to 8: stride, padding, the output-size formula, channels, and what filters learn.

3 What the guess buys, and when it does not pay

COVERS Lecture Part 3 · Lab Exercise 3

TIME 7 minutes

IN BRIEF Stacking stencils turns “nearby” into “everywhere”. The ending of the network decides whether the guess survives to the answer. And when the data was pre-arranged for you, the guess buys nothing.

WORDS IN THIS CHAPTER, IN PLAIN ENGLISH

layer	one round of stencils; the next layer works on the previous layer’s map
receptive field	how much of the original picture one output can “see”
pooling	shrink the map by keeping one number for each small block
invariance	move the picture and the answer does not change at all
the ending	the last step that turns a whole map into one answer, such as “cat”
pre-centred data	pictures where somebody already put the object in the middle for you

3.1 Depth turns “nearby” into “everywhere”

One layer of 3-by-3 stencils: each output sees 3 pixels across. Two layers: 5. Shrink the picture by half (a **pool**) and add two more: 14. Shrink again, two more: 24. By layer 8, one unit depends on all 1,024 pixels. Nobody wrote a rule about the whole picture; understanding of the whole was assembled from purely local looks. The growing window is the **receptive field**.

3.2 The ending can throw the guess away

Equivariance: move the input, the output moves. **Invariance**: move the input, the output does not change. You usually want invariance (a cat is a cat wherever it sits). The stencil layers give equivariance; whether you get invariance depends on the **ending** that turns the map into one answer.

same stencil layers, two endings	dials	positions seen in training	positions never seen
average the whole map	700	1.000	0.998
flatten, then wire every position	2,716	0.993	0.230

Trained with the pattern only on the left, tested only on the right. Averaging does not care where the pattern is. Flattening wired a separate dial to every position, so “on the right” is a place it has no trained dials for: chance. **A guess is worth only what your ending keeps of it.**

3.3 Where the guess fails, honestly

It fails when the data has no meaningful arrangement (a spreadsheet); when position genuinely matters (a shadow high in a lung is a different finding from one low down); when the important things are far apart; and when the change is not a shift (rotation, size). And the case nobody advertises: on small digit pictures that were **already centred and cropped**, the stencil model scored 0.940 and the plain model 0.978. Somebody solved “where is it?” before the model saw the data, so the guess had nothing left to buy.

COMMON MISTAKE

Believing a convolutional network is “better for images” in general. It is better when translation still needs solving. If the preprocessing already centred everything, the plain model can win, and that is the week’s point, not an exception to it.

★ FOR THE QUIZ

- Stacking layers turns local looks into a view of the whole picture: the **receptive field** grows.
- **Equivariance** (moves with the input) is not **invariance** (does not change). The **ending** decides: average the map, 0.998 on unseen positions; flatten and wire every position, 0.230.
- The guess fails when the data has no arrangement, when position matters, when things are far apart, or when the change is not a shift. On **pre-centred** data it buys nothing (0.978 vs 0.940).

EXTRA · OPTIONAL · NOT ON THE QUIZ

Full booklet Chapters 9 to 11: pooling, the CNN architectures and what each fixed, augmentation for rotation and scale.

4 Recurrence: one thing after another

COVERS Lecture Part 4 · Lab Exercise 4

TIME 9 minutes

IN BRIEF The same photocopied rulebook, applied along a sentence. A reader with a fixed-size notepad. Long chains forget because the signal is multiplied by the same number at every step; a gate that the network chooses fixes the forgetting, but not the notepad's size.

WORDS IN THIS CHAPTER, IN PLAIN ENGLISH

sequence	things that come one after another: words in a sentence, readings over time
recurrent network	a reader with a notepad: read a word, update the notepad, move on, same rulebook every time
hidden state	the notepad: a fixed number of slots, overwritten at every word
vanishing gradient	the signal from early words shrinking to nothing through repeated multiplication
gate	a dial between 0 and 1 that says how much of the memory to keep this step
LSTM	a recurrent network with a memory and gates

4.1 Sequences break three things at once

Length varies (three words or three hundred). Order carries the meaning (“the dog bit the man” and “the man bit the dog” use the same words). And the words that matter can be far apart: “The **keys** that I left on the table in the hall **are** missing.” A small stencil sitting on “are” sees “in”, “the”, “hall”; it cannot see “keys” at all.

4.2 One reader, one notepad

Picture a reader with a small notepad. Read a word; update the notepad using the word and what was already on it; move on. Same rulebook every time. At the end, answer from the notepad. That is a **recurrent** network; the notepad is called the **hidden state**. It is weight sharing again, turned from space to time. Two consequences: the notepad is a fixed size, and the answer depends on word 1 only through a chain of updates that may have written over it.

4.3 Why long chains forget

EVERYDAY ANALOGY

Try it with a pencil. Take 0.9 and multiply it by itself. $0.9 \times 0.9 = 0.81$. Again: 0.729. Again: 0.656. After ten times you are at 0.35; after thirty, 0.04; after sixty, 0.0018. Now do the same with 1.1: after ten times, 2.6; after thirty, 17; after sixty, 304. A tiny difference in the number, repeated sixty times, is the difference between nothing and an explosion. That is the whole of this section.

How much does word 1 still matter at word 60? The signal travels back through 60 identical updates, multiplied by **the same number** each time, so it behaves like one number raised to the power 60:

$$0.9 \text{ sixty times} = 0.0018 \qquad 1.1 \text{ sixty times} = 304$$

Two numbers 0.2 apart, results 170,000 times apart. An ordinary deep network has different numbers at each layer, so they can average out; a chain in time has one number, so there is nothing to average. This is the **vanishing (or exploding) gradient**, and it is arithmetic, not a bug.

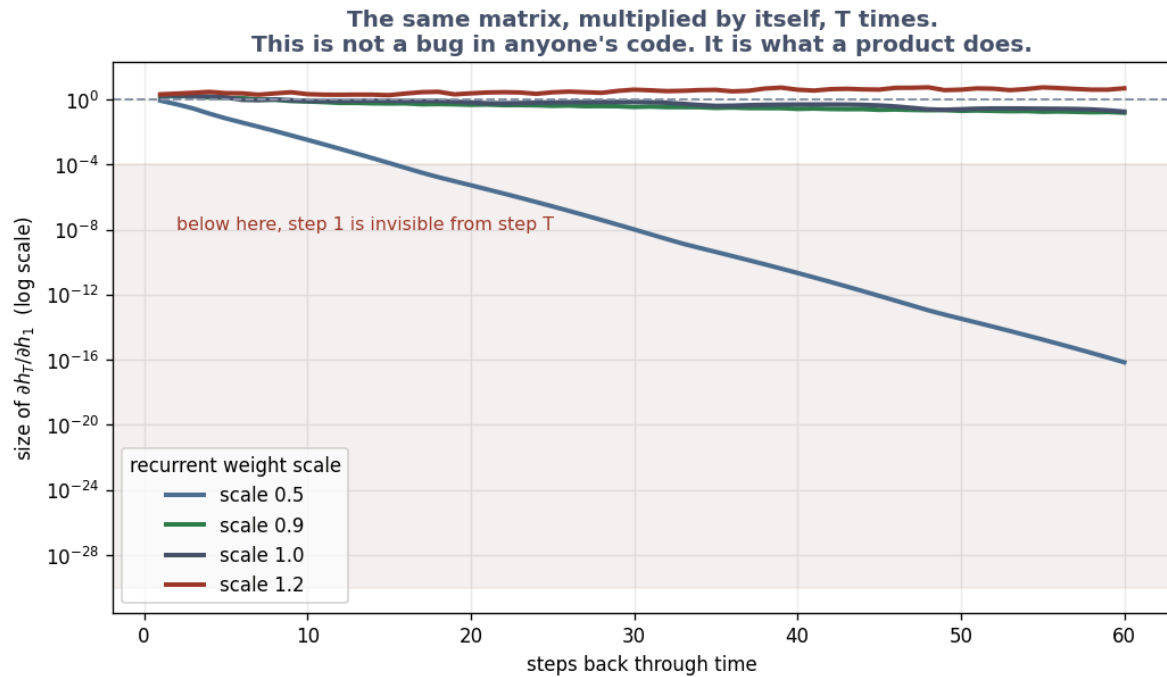


Figure 4: The signal from word 1, followed back through a real recurrent layer (from the lab). At weight scale 0.5 it is $7e-17$ after 60 steps: zero to a computer.

4.4 Gates: make forgetting a decision

The 1997 fix, the **LSTM**: give the reader a second notepad, the **memory**, with a **keep-gate** on it, a number between 0 (wipe) and 1 (keep everything), chosen fresh at every word. The old memory is **multiplied by the gate and added to**, never pushed through the rulebook and squashed. So the signal travelling back is multiplied only by the gate, and the network can learn to keep the gate near 1.

keep-gate	half the memory left after	1% left after
0.80	3 steps	21 steps
0.95	13 steps	90 steps
0.999	693 steps	4,603 steps

4.5 Gates fix the forgetting, not everything

words	plain chain: score	signal from word 1	with gates: score	signal from word 1
10	1.000	$1.9e-06$	1.000	$1.9e-08$
90	0.463	$7.8e-18$	1.000	$8.9e-06$
160	0.227	$1.6e-127$	0.250	$7.6e-08$

Row 90 is the argument: the plain chain's signal has collapsed and it is near chance; the gated reader's survives and it is perfect. Row 160 is the more useful lesson: the gated reader's signal **still survives** and it **still fails**. A surviving signal is necessary, not sufficient: the notepad is a fixed size, and 160 words' worth of "which was first?" did not fit. The second limit: the corridor runs one desk at a time, and modern chips are **wide**, not fast; a design that forbids doing things at once cannot use the chip. That, not accuracy, decided the field.

★ FOR THE QUIZ

- A **recurrent** network is one rulebook applied at every word, updating a fixed-size **notepad** (hidden state). Weight sharing along time.
- Long chains forget because the signal from early words is multiplied by **the same number** at every step (0.9 sixty times is 0.002; 1.1 sixty times is 304): the **vanishing / exploding gradient**.
- An **LSTM** adds a memory with a **keep-gate** chosen fresh each step; the memory is multiplied by the gate and added to, not squashed, so the signal can survive hundreds of steps.
- Two limits remain: the notepad is a **fixed size**, and the chain runs **one step at a time**.

EXTRA · OPTIONAL · NOT ON THE QUIZ

Full booklet Chapters 12 to 16: backpropagation through time written out, all four gates, the forget-gate bias trap, gradient clipping, GRUs.

5 Attention: who should talk to whom

COVERS Lecture Part 5 · Lab Exercise 5

TIME 9 minutes

IN BRIEF A noticeboard: every word says what it is and what it wants, and matching pairs talk directly. Four steps, one table whose rows add to 1, one surprising blindness, and a bill.

WORDS IN THIS CHAPTER, IN PLAIN ENGLISH

attention	every word looks at every other word and decides how much to listen to each
query, key, value	three cards each word writes: what I want, what I am, what I hand over
softmax	turn a row of scores into shares that add up to 1
position label	a stamp on each word saying where it sits, because attention cannot see order
head	one round of attention asking one question; several heads ask several
transformer	a machine built from blocks of attention plus a small ordinary network

5.1 A noticeboard, not a corridor

Each person in a room pins two cards: what they **are** (“I am a plumber”) and what they **want** (“I need a plumber”). Everyone reads everyone’s cards; matches talk directly; nothing is relayed. In a sentence, “are” pins “I am a plural verb; I want my subject” and “keys” pins “I am a plural noun”; they match across eight words in one step. The guess is much **weaker** than the other two: any word might matter to any other, and **content** decides which. Nothing about distance or order.

5.2 Every row adds up to 1

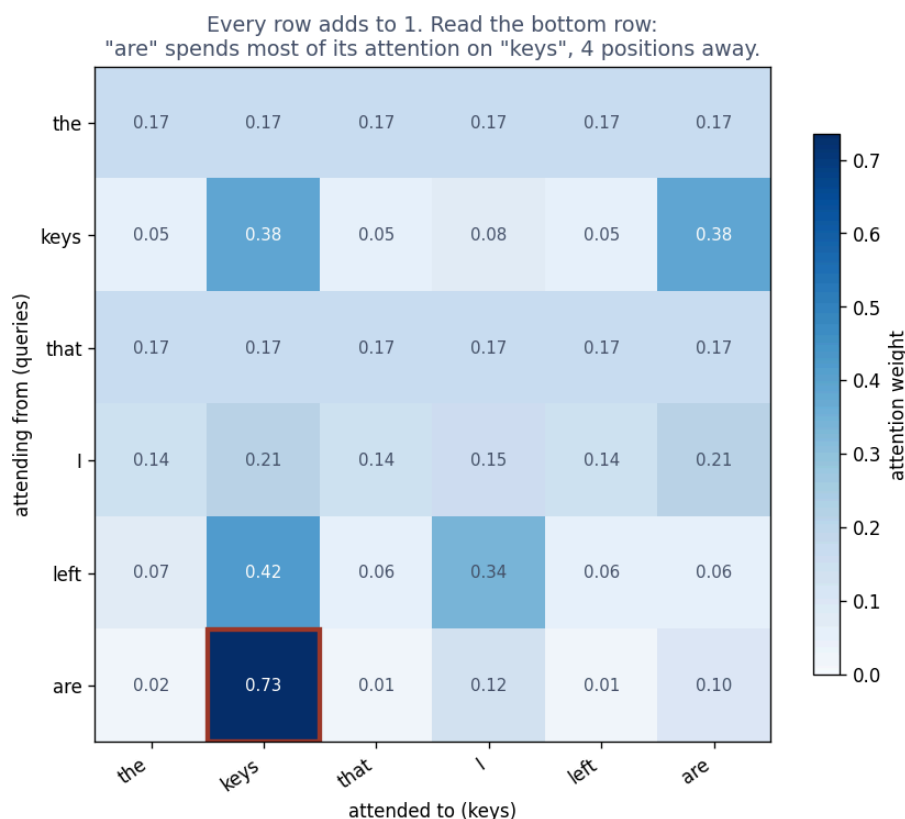


Figure 5: Six words, one table (from the lab). Each row is one word asking “who should I listen to?”; the numbers are shares that add up to 1. Row “are”: 0.735 on “keys”, four words away; 0.013 on “left”, next door.

5.3 The four instructions

EVERYDAY ANALOGY

A three-word example. Suppose the row of scores for the word “are” against the words “the”, “keys”, “left” comes out as 1, 5, 2. Softmax turns them into shares that add to 1: about 0.02, 0.93, 0.05 (the biggest score gets by far the biggest share). The new version of “are” is then 0.02 of what “the” hands over, plus 0.93 of what “keys” hands over, plus 0.05 of what “left” hands over. Almost all of it is “keys”. That is the whole mechanism, and nothing in it cared how far apart the words were.

- 1. Write the cards.** Each word writes a **query** (what I want), a **key** (what I am) and a **value** (what I hand over if chosen). Three small sets of dials do the writing.
- 2. Score.** Compare every query with every key; cards that point the same way score high.
- 3. Share out.** Turn each row of scores into shares that add up to 1 (**softmax**).
- 4. Mix.** Each word’s new version is the shares times the values: a blend of what the chosen words handed over.

In symbols, and it is the same four steps: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$. The $\sqrt{d}$ just keeps the scores a sensible size.

5.4 Attention cannot see order

Shuffle the words, run attention, un-shuffle the outputs, compare with the original order:

input	difference	verdict
-------	------------	---------

words only	8.88e-16	order is invisible: a bag of words
words + a position label	4.192	order now changes the answer

8.88e-16 is the computer's rounding error: the two runs are the same sums in a different order. Bare attention cannot tell “dog bites man” from “man bites dog”, so every transformer stamps a **position label** onto each word first: not a plain counter, which grows without limit, but a set of slow and fast **clocks** whose hand positions give every place a unique, small pattern.

5.5 Heads, the block, and the bill

One round of attention gives one blend, so a verb would have to ask for subject, object and tense in one question. Instead: split the width into several slices, attend separately in each, glue the results: several **heads**, several questions at once. A **transformer block** is attention (moves information **between** words) followed by a small ordinary network applied to each word alone (transforms information **within** a word), repeated a dozen times; about two thirds of the dials are in the small networks.

The bill: attention buys a path of length 1 between any two words, and pays in sums that grow with the **square** of the length. At 64 words it costs 1.1 times a recurrent chain; at 32,768 words, 65 times. That is why chatbots had short memories for years, and most fixes put a “nearby” guess back in.

★ FOR THE QUIZ

- **Attention** lets every word look at every other word directly; **content** decides who listens to whom. The guess is weak: nothing about distance or order.
- Four steps: write three cards per word (**query**, **key**, **value**); **score** every query against every key; **share out** each row so it sums to 1 (**softmax**); **mix** the values by those shares.
- Bare attention **cannot see order** (8.88e-16), so **position labels** are added first. Several **heads** ask several questions at once.
- A **transformer block** is attention (between words) then a small network (within a word), repeated. Cost grows with the **square** of the length.

EXTRA · OPTIONAL · NOT ON THE QUIZ

Full booklet Chapters 17 to 21: the softmax backward pass, sinusoidal position encodings, masking, why $\sqrt{d}$, and what the transformer gives up.

6 The bargain

COVERS Lecture Part 6 · Lab Exercise 6

TIME 5 minutes

IN BRIEF A guess built into the wiring substitutes for data. With little data the well-guessed machine wins; with enormous data the machine that guesses less learns the structure and then wins. Which applies to you is decided by how much data you have.

WORDS IN THIS CHAPTER, IN PLAIN ENGLISH

the bargain	a true guess in the wiring does the job that data would otherwise have to do
scale	training on enormous amounts of data and compute
Vision Transformer	a picture chopped into squares, fed to a transformer as if the squares were words
baseline	the simpler model you compare against, so you know what the clever one bought
crossover	the amount of data at which the machine that guesses less starts to win

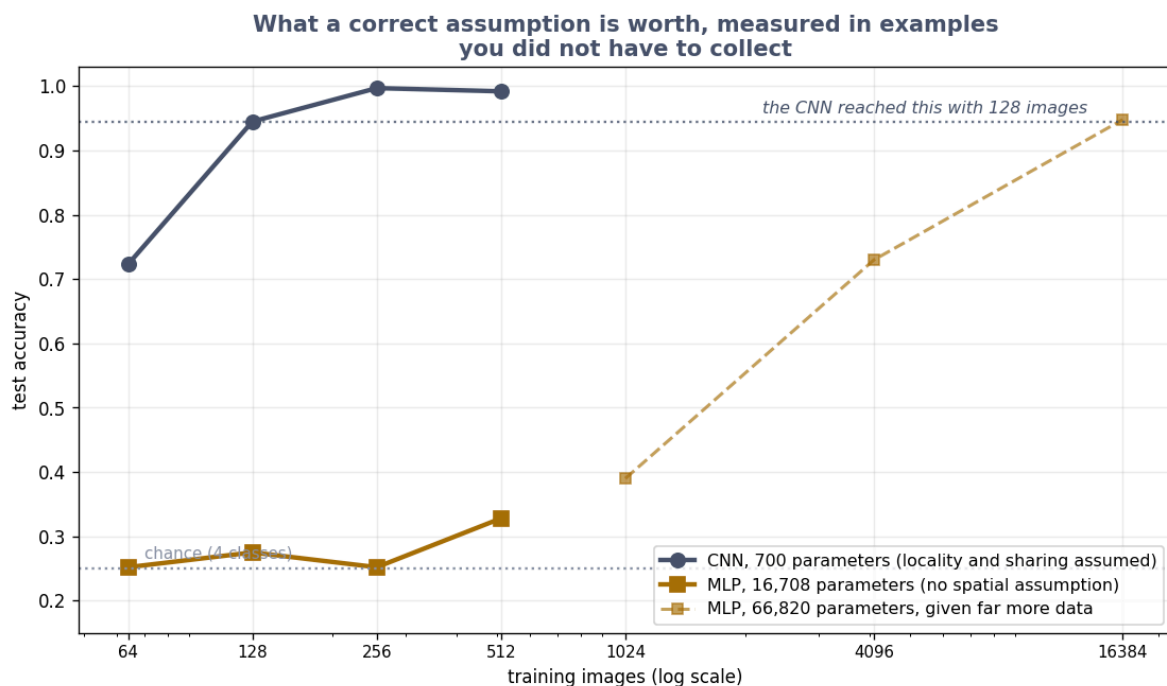


Figure 6: The bargain measured on a laptop (from the lab): the tiny well-guessed model against the big no-guess model as the data grows. The gap closes only when the big one has about 128 times the pictures.

6.1 The Vision Transformer settles it

Chop a photograph into small squares, treat the squares as words, run a transformer. Every guess about pictures is thrown away. Trained on 1.3 million pictures it **loses** to a stencil model; on 300 million it **beats** the same model. Same design, same task; only the data changed.

6.2 The famous essay, read properly

“The Bitter Lesson” observes that for seventy years methods using more computation have beaten methods that build in human knowledge. The misreading: “knowledge is useless, just make it bigger”. The careful reading is about **where the knowledge enters**: as a **rule about the answer** it tends to lose, because your rule is incomplete; as a **guess about the shape of the problem** (“nearby things belong together”) it is exactly the bargain, and it

wins whenever you do not have 300 million pictures. “Use more computation” is advice for organisations with hundreds of millions to spend; for everyone else, a true guess is the cheapest data you will ever get.

6.3 Seven questions for anything you design

1. What is connected to what, in one layer?
2. Which dials are shared, and across what: space, time, or nothing?
3. What does that wiring **assume**, said as a plain sentence about the world?
4. Is that sentence true of **my** data, and how would I know? (The shuffle test costs one line.)
5. Does my ending keep the guess all the way to the answer? (0.998 against 0.230.)
6. What does it cost, and where is the crossover?
7. How much data do I have, compared with what it would take to learn the structure from scratch?

★ FOR THE QUIZ

- **The bargain:** a true guess built into the wiring substitutes for data. Little data: the well-guessed machine wins. Enormous data: the machine that guesses less learns the structure and then wins.
- The **Vision Transformer** loses at 1.3 million pictures and wins at 300 million. Only the data changed.
- The **bitter lesson**, read properly, is about **where** knowledge enters: as a rule about the answer it loses; as a guess about the shape of the problem it is the bargain.
- Seven questions for any design, starting with: what is wired to what, and what does that assume?

EXTRA · OPTIONAL · NOT ON THE QUIZ

Full booklet Chapters 22 to 27: scaling laws, the resource argument, the same three families in PyTorch, and the responsibility angle.

7 The whole week on one page

COVERS Revision
TIME 3 minutes to read; 10 to memorise
IN BRIEF Every quiz-able statement from Week 4, in one place.

Topic	The sentence to be able to say
Inductive bias	Examples fit infinitely many rules; choosing one needs a preference not in the data. Every learner has one.
Wiring is a guess	Which inputs may influence which outputs is a claim about the world. True: fewer dials, less data. False: worse than no guess.
Flexibility	A more flexible model is not a better model. 700 dials beat 16,708 by 67 points; the big one had 1.000 on training.
Shuffle test	Destroy only what the guess is about. The tiny model fell to chance; the big one did not move.
Convolution	A kernel slid over the picture: multiply, add, write one number. The kernel never changes: weight sharing.
Three assumptions	Locality · stationarity · compositionality.
Equivariance	Shift the picture, the output shifts the same way, exactly. Invariance: the output does not change. The ending decides (0.998 vs 0.230).
Receptive field	Stacked layers see further; by layer 8 one unit sees all 1,024 pixels.
When it fails	No arrangement · position matters · far apart · not a shift · pre-centred data (0.978 vs 0.940).
Recurrence	One rulebook at every word, updating a fixed-size notepad.
Forgetting	The same number at every step: 0.9 sixty times is 0.002; 1.1 sixty times is 304.
LSTM	A memory with a keep-gate chosen each step; multiplied and added, not squashed.
Two limits	Fixed notepad · one step at a time.
Attention	Every word looks at every word; content decides. Query, key, value; score; softmax; mix.
Order	Bare attention cannot see it (8.88e-16); position labels are added.
Heads, block, bill	Several questions at once · attention between words then a small network within · cost grows with the square.
The bargain	A guess substitutes for data; which wins depends on how much data you have. Vision Transformer: 1.3M loses, 300M wins.

7.1 Twelve words, in plain English

inductive bias	the preference a learner uses to pick one rule from the many that fit
architecture / wiring	which parts of the input may influence which outputs
kernel	the small square of numbers slid over a picture
weight sharing	the same numbers used at every position (or every step in time)

feature map	the output of a kernel: a map of where its pattern was found
equivariance / invariance	output moves with the input / output does not change
receptive field	how much of the picture one output can see
hidden state	the reader's fixed-size notepad
vanishing gradient	the signal from early words shrinking to nothing through repeated multiplication
gate	a chosen number between 0 and 1 that says how much memory to keep
query, key, value	what a word wants, what it is, what it hands over
softmax	turn a row of scores into shares that add up to 1

8 Twenty practice questions, with answers

COVERS Quiz preparation

TIME 20 minutes

IN BRIEF Cover the right-hand column, answer, uncover. Every question is one of the eight quiz topics in disguise.

#	Question	Answer
1	Why can three examples never determine the fourth answer on their own?	Infinitely many rules fit any finite set of examples; a preference (inductive bias) is needed to choose.
2	What does it mean to say an architecture is an assumption?	Restricting which inputs may influence which outputs asserts something about the data, e.g. that nearby things are related.
3	A model with 24 times the dials scored at chance where a tiny one scored 0.945. Was it undertrained?	No; it scored 1.000 on training. It lacked a reason to prefer the right answer.
4	What does the shuffle test destroy, and what does it keep?	It destroys which pixels are neighbours; it keeps every pixel and every label.
5	Describe a convolution in one sentence.	Slide a small square of numbers over the picture; at each position multiply the pairs and add; the square never changes.
6	Why does an edge kernel give zero on a flat patch?	Its nine numbers sum to zero, so any constant patch multiplies out to zero.
7	Name the three assumptions of a convolution.	Locality, stationarity (weight sharing), compositionality.
8	What is equivariance, and why is a convolution's error exactly zero?	Shift the input and the output shifts the same way; the same numbers are used everywhere, so the operation cannot tell where it is.
9	Equivariance vs invariance, in one line each.	Equivariance: output moves with the input. Invariance: output does not change.
10	Two CNNs with identical convolution layers score 0.998 and 0.230 on unseen positions. What differed?	The ending: averaging the map keeps the guess; flattening and wiring every position throws it away.
11	Give a case where a CNN's guess buys nothing.	Pre-centred, pre-cropped digits: translation was already solved; the plain model won 0.978 vs 0.940.
12	What is the hidden state of a recurrent network, in plain words?	A fixed-size notepad, updated at every word by the same rulebook.
13	Why does a plain recurrent chain forget the first word?	The signal is multiplied by the same number at every step, so it behaves like that number to the power of the length: 0.9 sixty times is 0.002.
14	What does an LSTM's keep-gate change?	The memory is multiplied by a gate the network chooses (near 1 to keep) and added to, not squashed, so the signal survives.

15	At 160 words the LSTM's signal survives and it still fails. What does that show?	A surviving signal is necessary, not sufficient; the fixed-size notepad is the other limit.
16	Why did hardware, not accuracy, decide against recurrence?	The chain runs one step at a time; modern chips are wide and need many sums at once.
17	The four steps of attention.	Write query, key and value cards; score every query against every key; softmax each row to shares summing to 1; mix the values by the shares.
18	What does 8.88e-16 in the shuffle test mean?	Rounding error: bare attention gives the same output for shuffled words. It cannot see order; position labels are added.
19	What does attention cost as the sequence grows?	Sums grow with the square of the length: 65 times a recurrent chain at 32,768 words.
20	State the bargain, and the Vision Transformer's evidence.	A built-in guess substitutes for data; with little data the guessed machine wins, with enormous data the other wins. ViT loses at 1.3M pictures, wins at 300M.